

SEO-DBSCAN: Securely and Efficiently Outsourcing Density-Based Clustering

Ke Cheng , Xinghui Zhu , Jiaxuan Fu, Zhiwei Zhang , Jian Yang , Aijing Sun ,
Haichang Gao , *Member, IEEE*, and Yulong Shen , *Senior Member, IEEE*

Abstract—DBSCAN, a well-known density-based clustering algorithm, offers distinct advantages in identifying clusters of arbitrary shapes, outperforming traditional partitioning and hierarchical methods. However, when outsourcing DBSCAN computations to the cloud, data owners face substantial privacy risks due to the sensitivity of their input data. Many existing secure DBSCAN schemes face challenges such as reduced clustering accuracy, unintentional privacy leakage, and limited scalability. In this paper, we present SEO-DBSCAN, a secure and efficient outsourced computational framework specifically tailored for the DBSCAN algorithm. SEO-DBSCAN achieves clustering quality on par with plaintext execution, while ensuring strong privacy guarantees and high efficiency. Two key innovations drive its performance: 1) a constant-round secure comparison protocol built entirely upon lightweight arithmetic secret sharing, enabling efficient neighborhood queries and cluster formation; and 2) a crypto-friendly variant of DBSCAN, redesigned to support parallelizable and computation-efficient secure processing. Leveraging these techniques, SEO-DBSCAN significantly reduces the overhead of secure clustering in the cloud, particularly in terms of communication rounds. Experimental results on synthetic datasets show that SEO-DBSCAN achieves up to $8.65 \times$ computational savings and $83.7 \times$ communication reduction compared to ppDBSCAN (AsiaCCS'21).

Received 5 July 2025; revised 1 April 2026; accepted 4 July 2026. Date of publication 8 July 2026; date of current version 1 September 2026. This work was supported in part by the Key R&D Program of Shandong Province of China under Grant 2023CXPT056, in part by the National Natural Science Foundation of China under Grant 62402358, Grant 62220106004, Grant 62502363, and Grant 62572371, in part by the National Key R&D Program of China under Grant 2023YFB3107500, in part by Young Elite Scientist Sponsorship program by CAST under Grant YESS20240717, in part by the Young Talent Fund of Association for Science and Technology in Shaanxi, China under Grant 20240138, in part by the Key Research and Development Program of Shaanxi under Program 2025PT-ZCK-52, in part by the Open Topics from the Lion Rock Labs of Cyberspace Security under Project #LRL24004, in part by the Fundamental Research Funds for the Central Universities under Grant ZDRC2202, and in part by the Xidian University Specially Funded Project for Interdisciplinary Exploration under Grant TZJHF202502. (*Corresponding author: Xinghui Zhu.*)

Ke Cheng, Xinghui Zhu, Zhiwei Zhang, Haichang Gao, and Yulong Shen are with the School of Computer Science and Technology, Xidian University, Xi'an 710126, China, and also with the Shaanxi Key Laboratory of Network and System Security, Xi'an 710071, China (e-mail: chengke@xidian.edu.cn; xhzhu@xidian.edu.cn; zwzhang@xidian.edu.cn; hchgao@xidian.edu.cn; ylshen@mail.xidian.edu.cn).

Jiaxuan Fu is with Tencent Inc., Shenzhen 518054, China (e-mail: jiaxuanfu@tencent.com).

Jian Yang is with China Electronics Data Corporation, Shenzhen 518057, China (e-mail: ferryjian@gmail.com).

Aijing Sun is with the School of Communication and Information Engineering, Xi'an University of Posts and Telecommunications, Xi'an 710121, China, and also with the Shaanxi Key Laboratory of Information Communication Network and Security, Xi'an University of Posts and Telecommunications, Xi'an 710121, China (e-mail: sunaijing@xupt.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2026.3711460>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3711460

Moreover, evaluations on real-world datasets demonstrate that SEO-DBSCAN consistently outperforms FSS-DBSCAN (TIFS'24) and PPA-DBSCAN (TDSC'24) in both runtime and communication cost, highlighting its superior scalability and practical applicability in large-scale secure clustering scenarios.

Index Terms—Secure clustering, DBSCAN, additive secret sharing, secure comparison.

I. INTRODUCTION

CLUSTERING is a core unsupervised learning technique for analyzing large-scale unlabeled data [1]. To handle the growing computational and storage demands, clustering tasks are increasingly outsourced to cloud platforms. However, since cloud servers are not fully trusted and clustering often involves sensitive data from multiple parties, secure outsourced clustering protocols are essential to prevent privacy leakage [2], [3]. Existing secure clustering research has primarily focused on K-means [4], [5], [6], which is limited to convex clusters and sensitive to noise. DBSCAN [7] offers greater robustness and flexibility, but current secure DBSCAN schemes still leak sensitive information such as cluster assignments or cluster sizes [8], [9]. Such leakage can expose population-level insights in collaborative settings, highlighting the need for secure DBSCAN protocols that protect both individual data and global clustering statistics.

Fully private outsourced DBSCAN clustering refers to a setting that hides raw data, intermediate computation results, and sensitive clustering statistics while revealing only the prescribed final output, yet this setting remains largely underexplored. ppDBSCAN [10], the first secure outsourced DBSCAN scheme, relies on the ABY framework [11] and enforces a fixed upper bound on clustering iterations, which is difficult to determine in practice and leads to prohibitive runtime (e.g., up to one week for 5,000 two-dimensional points). PPA-DBSCAN [12] improves efficiency via ρ -approximate clustering [13], but sacrifices accuracy, requires sensitive hyperparameter tuning, and depends on costly secure set operations. FSS-DBSCAN [14] further reduces online rounds using function secret sharing, yet incurs substantial setup overhead due to key generation and distribution, resulting in high end-to-end cost.

Designing an efficient fully private outsourced DBSCAN protocol is challenging for at least two reasons. First, secure comparison is a fundamental building block in DBSCAN, yet existing solutions incur either high computation overhead or high round complexity. For example, ppDBSCAN [10] relies

on garbled circuits, which are computationally expensive, while FSS-DBSCAN uses an FSS-based three-party comparison protocol that requires five rounds of interaction for each comparison. Since comparison is invoked repeatedly throughout clustering, such costs become a major performance bottleneck. Second, DBSCAN itself is inherently iterative and data-dependent: cluster expansion relies on queue-based density connectivity, and the execution path depends on the neighborhood structure of the input data. Directly securing this process may leak sensitive access patterns or neighborhood relations. Enforcing full privacy therefore requires a data-oblivious reformulation of DBSCAN, but naive approaches are prohibitively expensive. For instance, ppDBSCAN obscures data-dependent behavior using triple-nested loops, resulting in $O(N^3)$ complexity for N points, while PPA-DBSCAN improves efficiency only by introducing approximation and still depends on costly secure set operations. Therefore, *the key challenge is not merely to secret-share the data, but to jointly design lightweight secure primitives and a crypto-friendly DBSCAN workflow that preserves privacy, accuracy, and scalability at the same time.*

To address the above challenges, we present SEO-DBSCAN, a secure and efficient outsourced density-based clustering framework built on a two-cloud, non-colluding architecture. Rather than directly applying secret sharing to the standard DBSCAN workflow, SEO-DBSCAN combines lightweight arithmetic secret sharing with a crypto-friendly reformulation of DBSCAN. At the cryptographic level, we design a constant-round secure comparison (SC) protocol based on LSB extraction and plaintext less-than testing, which achieves only four communication rounds independent of input bit-length. At the algorithmic level, we decompose DBSCAN into a parallelizable neighborhood query phase and a data-oblivious clustering phase. By eliminating data-dependent control flow and restructuring the algorithm into arithmetic-only iterations, our crypto-friendly DBSCAN variant supports efficient secure execution with quadratic scalability, while preserving clustering accuracy comparable to the plaintext algorithm.

Our main contributions are summarized as follows:

- We design a suite of lightweight cryptographic primitives optimized for parallel computation, and use them to construct a constant-round, communication-efficient secure two-party comparison protocol. These sub-protocols are generic and may be of independent interest for broader secure applications.
- Along with the above cryptographic protocols and our tailored crypto-friendly DBSCAN variant, we propose SEO-DBSCAN, an outsourced DBSCAN clustering scheme, which achieves the same clustering quality as plaintext computation, full privacy protection, and state-of-the-art efficiency at the same time.
- We implement a system prototype and evaluate SEO-DBSCAN through extensive experiments on both simulated and real-world datasets. Results on simulated data show that SEO-DBSCAN achieves up to $8.65\times$ speedup in computation and $83.7\times$ reduction in communication compared to ppDBSCAN. On real-world datasets, SEO-DBSCAN consistently outperforms state-of-the-art

baselines in runtime and communication efficiency, while preserving clustering accuracy, demonstrating its strong scalability and practical viability for large-scale privacy-preserving clustering tasks.

The rest of the paper is organized as follows. Section II presents the related work. Section III and Section IV introduce preliminaries and system overview. The building blocks are introduced in Section V. The details of the secure clustering scheme are presented in Section VI. The theoretical analysis and experimental evaluations are shown in Section VII and Section VIII. Finally, Section IX concludes the paper.

II. RELATED WORK

A. Existing Secure DBSCAN-Based Clustering Schemes

A number of works have studied privacy-preserving DBSCAN clustering over distributed or encrypted data. Early approaches focus on vertically or horizontally partitioned datasets using homomorphic encryption or secure multiparty protocols [15], [16], [17]. While these schemes enable clustering on protected data, they often leak sensitive intermediate information such as neighborhood relations or cluster sizes. To reduce such leakage, Anikin et al. [18] combine homomorphic encryption with additive masking under a semi-trusted model, but their approach incurs high computational overhead and requires all parties to remain online, making it unsuitable for outsourced settings. More recent works explore outsourced DBSCAN clustering [8], [9], yet still reveal sensitive information, such as cluster sizes or assignment patterns, due to design choices like order-preserving encryption. These limitations highlight the challenge of achieving efficient and fully private outsourced DBSCAN clustering.

Recently, Bozdemir et al. [10] design a secure DBSCAN clustering protocol with full privacy (denoted as ppDBSCAN-full) for the first time. Their approach strictly adheres to the original DBSCAN algorithm, thus achieving the same accuracy as plaintext. However, ppDBSCAN is realized with a triple-loop structure to ensure full privacy preservation. This means that ppDBSCAN achieves secure clustering through $O(N^3)$ complexity in terms of the number of input points N , which is impractical over large datasets. This work also proposes an approximation algorithm to reduce the complexity of their method to $O(N^2)$, which causes additional information leaks that reveal how the input data is distributed. Moreover, despite the security compromises, ppDBSCAN takes over a week to securely cluster 5000 2-dimensional points.

To improve efficiency, PPA-DBSCAN [12] proposes an approximate DBSCAN protocol based on the concept of ρ -approximate clustering [13]. Although this approach reduces computational cost, it introduces a loss in clustering accuracy and exhibits high sensitivity to hyperparameter tuning, which is particularly challenging in encrypted settings. In addition, the protocol involves complex set-based operations such as union-find structures that are computationally expensive to implement securely without leaking private information. FSS-DBSCAN [14] represents the state-of-the-art in secure three-party DBSCAN outsourcing by leveraging function secret

sharing (FSS) to minimize round complexity and improve protocol responsiveness. Despite its advantages in online phase efficiency, the scheme incurs considerable overhead during the setup phase due to the costly generation and distribution of FSS keys. This initialization burden contributes to a relatively high overall end-to-end cost, which limits its practical scalability in some real-world deployments.

In summary, secure clustering solutions with full privacy have been rarely studied, and even the most advanced methods still struggle to balance security and execution efficiency. Given these challenges, we aim to design a secure outsourced clustering scheme that is fast in computation and efficient in communication.

B. Existing Secure Comparison Protocols

Secure comparison is a fundamental building block for secure clustering and broader privacy-preserving computation. Since Yao's seminal "Millionaire's Problem" [19], a wide range of secure comparison protocols have been proposed. We refer interested readers to the comprehensive survey by Veugen et al. [20] for a detailed review. In this section, we focus on recent advances relevant to secret-sharing-based approaches. As highlighted by Veugen et al. secret-sharing-based comparison protocols [21], [22] often offer superior efficiency in the online phase compared to those based on garbled circuits or homomorphic encryption. Zheng et al. [23] reduce the comparison task to extracting the most significant bit (MSB) from secret-shared values, implemented using a full-adder logic with ℓ rounds for ℓ -bit inputs. Liu et al. [24] improve upon this with a parallel prefix adder design, reducing the round complexity to $\log \ell$, though still not constant. Morita et al. [25] propose a constant-round protocol using multi-fan-in gates, requiring 6 rounds. Ohata et al. [26] further reduce the round complexity via a Beaver triple extension mechanism, but this approach incurs high local computation and memory costs, often dominating runtime. Motivated by these limitations, we propose a new secure comparison protocol that combines 0/1 encoding with additive secret sharing. Our design achieves a favorable trade-off between computational overhead and communication rounds, resulting in improved efficiency for secure clustering workloads.

III. PRELIMINARIES

A. DBSCAN Clustering Algorithm

DBSCAN [7] is a popular density-based clustering algorithm, which is defined via two parameters: ϵ (*eps*) and *MinPts*. Some key definitions of DBSCAN are as follows.

Definition 1: The ϵ -**neighborhood** of a point p , denoted by $N_\epsilon(p)$, is defined by $N_\epsilon(p) = \{q | q \in P, \text{dist}(p, q) \leq \epsilon\}$, where P is a set of points and $\text{dist}(p, q)$ is the Euclidian distance between p and q .

Definition 2: A point $p \in P$ is a **core point** if $N_\epsilon(p)$ contains at least *MinPts* points of P (including p itself).

Definition 3: A point $q \in P$ is **density-reachable** from $p \in P$ if there is a chain of points $p_1, p_2, \dots, p_t$ such that:

- $p_1 = p$ and $p_t = q$

- $p_1, p_2, \dots, p_{t-1}$ are core points
- $p_{i+1} \in N_\epsilon(p_i)$ for each $i \in [1, t-1]$.

Definition 4: A point $p \in P$ is **density-connected** to a point $q \in P$ if there is a point o such that both p and q are density-reachable from o .

Definition 5: A **cluster** C with respect to ϵ and *MinPts* is a non-empty subset of P such that:

- (**Maximality**) If a core point $p \in C$, then all the points density-reachable from p also belong to C .
- (**Connectivity**) For any points $p, q \in C$, p is density-connected to q .

Remark that a non-core point $p \in P$ in a cluster is called a **border point**. A point $p \in P$ is a **noise point** if it does not belong to any cluster.

The DBSCAN algorithm is employed to discover a unique set of clusters over the dataset P . It begins with an arbitrary unlabeled point and iteratively identifies clusters by selecting unlabeled core points and expanding them through density-reachable neighbors. This process continues until all core points have been assigned to a cluster. The detailed steps of DBSCAN are provided in Algorithm 7 in Appendix A, available online.

B. Cryptographic Primitives

1) Arithmetic Secret Sharing: As a basic terminology throughout the paper, we let p be an odd prime and ℓ be the bit length of p , i.e., $\ell = \lceil \log_2 p \rceil$. In the following, we assume all arithmetic operations to be performed in the odd field $\mathbb{Z}_p = \{0, \dots, p-1\}$, i.e., all operations are (mod p). A 2-out-of-2 arithmetic secret sharing scheme [27] over $\mathbb{Z}_p$ consists of two algorithms: Share and Reconst, which are defined as follows:

- **Share(x):** Let P_i be the party holding the plaintext input x . Party P_i samples a random value $r \xleftarrow{\$} \mathbb{Z}_p$, sets its local share as $\langle x \rangle_i = x - r \bmod p$, and sends r to the other party P_{1-i} , who sets $\langle x \rangle_{1-i} = r$. The resulting arithmetic sharing is denoted as $\langle x \rangle = (\langle x \rangle_0, \langle x \rangle_1)$.
- **Reconst($\langle x \rangle$):** Party P_i sends $\langle x \rangle_i$ to party P_{1-i} who computes $x = \langle x \rangle_i + \langle x \rangle_{1-i}$.

Next, we introduce secure protocols for fundamental operations over the above secret-shared data. ADD takes as input two shared values $\langle x \rangle, \langle y \rangle$, and outputs $\langle x + y \rangle$ without revealing the real values of x, y . MUL takes as input two shared values $\langle x \rangle, \langle y \rangle$, and outputs $\langle xy \rangle$ without revealing the real values of x, y . These operations are defined as follows:

- **ADD($\langle x \rangle, \langle y \rangle$):** P_i locally computes $\langle x + y \rangle_i \leftarrow \langle x \rangle_i + \langle y \rangle_i$ (for $i \in \{0, 1\}$).
- **MUL($\langle x \rangle, \langle y \rangle$):** We will use Beaver's multiplicative triples [28] of the form $\{\langle a \rangle, \langle b \rangle, \langle ab \rangle\}$, which are shared among the two parties. P_i locally computes $\langle e \rangle_i \leftarrow \langle x \rangle_i - \langle a \rangle_i$ and $\langle f \rangle_i \leftarrow \langle y \rangle_i - \langle b \rangle_i$. Both parties perform **Reconst($\langle e \rangle$)** and **Reconst($\langle f \rangle$)**, then set $\langle xy \rangle_i \leftarrow i \cdot ef + e \cdot \langle b \rangle_i + f \cdot \langle a \rangle_i + \langle ab \rangle_i$.

We write the ADD and MUL protocols simply as $\langle x + y \rangle \leftarrow \langle x \rangle + \langle y \rangle$ and $\langle xy \rangle \leftarrow \langle x \rangle \cdot \langle y \rangle$, respectively. In particular, the addition and multiplication can also be applied to a secret-shared value $\langle x \rangle$ and a public constant c , and we write $\langle x + c \rangle \leftarrow \langle x \rangle + c$ and $\langle cx \rangle \leftarrow c \cdot \langle x \rangle$. Note that Beaver's multiplicative

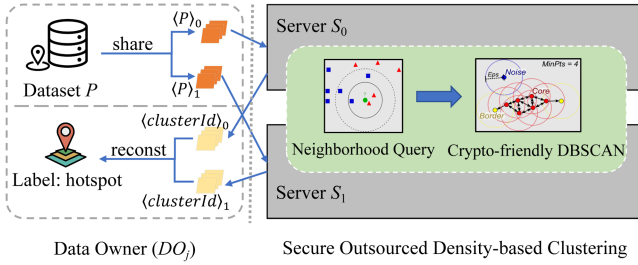


Fig. 1. System model (The green components illustrate secure collaboration between servers S_0 and S_1 based on secret sharing. All data exchanged between the two cloud servers and the data owners is also maintained in secret-shared form to preserve privacy throughout the protocol.).

triples can be generated in an offline stage, via homomorphic encryption or correlated oblivious transfer (COT). Therefore, we assume that the triples are pre-computed and available for use in our scheme.

2) *Constant-Round Multi-Fan-In and (AND*) Protocol*: The functionality AND* takes as input n arithmetic shared values $\langle x_1 \rangle, \langle x_2 \rangle, \dots, \langle x_n \rangle$ over $\mathbb{Z}_p$ and returns arithmetic shares of $\langle x_1 \wedge x_2 \wedge \dots \wedge x_n \rangle$, where $x_i \in \{0, 1\}$ for all $i \in [1, n]$. We use the communication-efficient secure two-party protocol proposed by Morita et al. [25] to achieve this functionality, and it only needs 2 rounds and $2n$ field elements for communications. We refer the reader to [25] for details.

C. 0-Encoding and 1-Encoding

Our scheme uses two special encodings, 0-encoding, and 1-encoding [29], which can reduce the less than problem to the equality test problem. Let $s = s_{\ell} s_{\ell-1} \dots s_1 \in \{0, 1\}^\ell$ be an ℓ -length binary string. The 0-encoding of s is defined as a set S_s^0 such that

$$S_s^0 = \{s_{\ell} s_{\ell-1} \dots s_{i+1} 1 \mid s_i = 0, 1 \leq i \leq \ell\}$$

The 1-encoding of s is the set S_s^1 such that

$$S_s^1 = \{s_{\ell} s_{\ell-1} \dots s_i \mid s_i = 1, 1 \leq i \leq \ell\}$$

Both S_s^0 and S_s^1 have at most ℓ elements.

To compare two ℓ -bit integers x and y , we encode x into 0-encoding S_x^0 and y into 1-encoding S_y^1 . Then, we can make the judgment that $x < y$ by the following formula:

$$x < y \iff |S_x^0 \cap S_y^1| = 1. \quad (1)$$

Lin et al. [29] have proved (1) clearly. Here we give an example to explain it. Let $x = 6 = 0110_2$ and $y = 10 = 1010_2$ of length 4. We have $S_x^0 = \{1, 0111\}$ and $S_y^1 = \{1, 101\}$. Since $|S_x^0 \cap S_y^1| = 1$, we have $x < y$ indeed. If $x = 10 = 1010_2$ and $y = 6 = 0110_2$, we have $S_x^0 = \{11, 1011\}$ and $S_y^1 = \{01, 011\}$. Since $|S_x^0 \cap S_y^1| \neq 1$, we have $x \geq y$.

IV. SYSTEM OVERVIEW

A. System Model

Fig. 1 illustrates the architecture of the proposed secure outsourced density-based clustering system, which consists of multiple data owners (DOs) and two non-colluding cloud servers,

denoted by S_0 and S_1 . Each data owner DO_j holds a private local dataset $P^{(j)} = \{p_1^{(j)}, \dots, p_{n_j}^{(j)}\}$, where each point contains its spatial features (e.g., geographic coordinates). As shown in Fig. 1, the symbol “ P ” refers to these input data points. To protect data confidentiality, every owner secret-shares its dataset into two additive shares, $\langle P^{(j)} \rangle_0$ and $\langle P^{(j)} \rangle_1$, and uploads them to S_0 and S_1 , respectively. Each share reveals no information about the underlying values.

Given the global secret-shared dataset $P = \bigcup_j P^{(j)}$, the two servers jointly execute a secure density-based clustering protocol. As highlighted by the green components in Fig. 1, the protocol includes (i) a secure neighborhood-query procedure that evaluates distance comparisons on secret shares, and (ii) a crypto-friendly DBSCAN procedure that securely determines core points, propagates density reachability, and forms cluster structures. All intermediate states, including distances, neighbor relationships, core indicators, and reachability flags, remain secret-shared throughout computation.

Upon completion, the servers obtain secret shares of the clustering output. Specifically, each point is assigned a cluster identifier, denoted as “clusterID” in Fig. 1. The servers return $\langle \text{clusterID}^{(j)} \rangle_0$ and $\langle \text{clusterID}^{(j)} \rangle_1$ to the corresponding data owner, who locally reconstructs clusterID^(j) and retrieves its final clustering result. Neither server learns any information about the raw input data, the clustering structure, or the resulting labels.

B. Threat Model

We address the problem of secure outsourced density-based clustering under the semi-honest adversary model. In this setting, all participants follow the protocol faithfully but may attempt to infer additional information from the messages they receive during execution. We further assume that the two cloud servers are non-colluding, a standard assumption in prior works [12], [30], [31]. This assumption is justified in practice, as the servers are typically operated by independent or competing service providers, and collusion would pose reputational and economic risks that deter such behavior.

The non-colluding two-cloud model has been widely adopted in both theoretical and practical secure computation systems. Many works [12], [14], [24], [30], [31] employ two independent cloud servers that collaboratively perform various computational tasks while preserving input/output privacy, ensuring that no single party learns any private data. A real-world business example is Partisia [32], which builds its MPC platform on two or more non-colluding servers and has commercially deployed such systems since 2008, initially supporting secure auctions such as energy and spectrum license trading. These deployments clearly validate the practicality and real-world viability of the two-cloud-server model for secure computation.

C. Design Goals

Our scheme will fulfill the following design goals:

- *Correctness*: The clustering results should be consistent with those produced by the plaintext DBSCAN algorithm, which is known for its robustness across diverse datasets

and its ability to accurately detect arbitrarily shaped clusters and noise.

- **Security:** The protocol must guarantee strong privacy protection under the semi-honest adversary model. Specifically, it should reveal no information beyond the final clustering output—this includes suppressing leakage of intermediate values such as cluster sizes, neighborhood relationships, or distance comparisons. All sub-protocols should be provably secure within this threat model.
- **Efficiency:** The scheme should be communication- and computation-efficient, ensuring practical performance even on large-scale datasets. It should support partial parallelization for scalability and allow data owners to remain offline after submitting their secret shares.

V. CONSTANT-ROUND SECURE COMPARISON PROTOCOLS

In this section, we present a constant-round secure comparison (SecureComparison) protocol on secret-shared data, composed of three generic sub-protocols. We assume the existence of two non-colluding parties (P_0 and P_1), and our protocols are realized via the interaction of the two parties. The specific protocols are described as follows.

- **BinaryStringEqual:** secure equality test functionality with plain binary string inputs.
- **PlainLessThan:** secure less-than comparison functionality with plain inputs
- **ComputeLSB:** secure least-significant-bit computation functionality with arithmetic shared inputs.
- **SecureCompare:** secure comparison functionality with arithmetic shared inputs.

A. Binary String Equal Protocol

An binary string equal protocol $\text{BSE}(x, y)$ outputs $\langle \delta \rangle$ with plain binary string inputs x, y , where $\delta = 1$ iff $x = y$. Note that $x = x_{\{\ell\}}x_{\{\ell-1\}} \cdots x_{\{1\}} \in \{0, 1\}^\ell$ and $y = y_{\{\ell\}}y_{\{\ell-1\}} \cdots y_{\{1\}} \in \{0, 1\}^\ell$. To determine whether or not two strings are equal, we check if the i -th bit of x ($x_{\{i\}}$) equals the i -th bit of y ($y_{\{i\}}$) for each $i \in [1, \ell]$. Specifically, P_0 sets $\langle x_i \rangle_0 = x_{\{i\}}$, $\langle y_i \rangle_0 = 0$, and P_1 sets $\langle x_i \rangle_1 = 0$, $\langle y_i \rangle_1 = y_{\{i\}}$. In this way, P_0 and P_1 get the shares of each bit of x, y without any communication cost. Then, P_0 and P_1 computes $\langle f_i \rangle = (1 - \langle x_i \rangle - \langle y_i \rangle)^2$, where $f_i = 1$ if $x_i = y_i$ and $f_i = 0$ otherwise. At last, P_0 and P_1 invoke the AND^* protocol [25] to compute $\langle \delta \rangle = \text{AND}^*(\langle f_1 \rangle, \langle f_2 \rangle, \dots, \langle f_\ell \rangle) = \langle f_1 f_2 \cdots f_\ell \rangle$. Obviously we obtain that $\delta = 1$ iff $f_i = 1$ for all $i \in [0, \ell]$. The corresponding protocol is described in Algorithm 1.

B. Plain Less Than Protocol

A less-than comparison protocol $\text{PlainLessThan}(x, y)$ outputs $\langle \delta \rangle$ with plain inputs $x, y \in \mathbb{Z}_p$ ($\ell = \lceil \log_2 p \rceil$), where $\delta = 1$ iff $x < y$. PlainLessThan protocol is based on 0-encoding and 1-encoding introduced in Section III-C, which is tractable for parallel computation (and hence requires small constant rounds of communication). Recall that this approach can convert a less-than operation to multiple parallel equality test operations

Algorithm 1: BinaryStringEqual Protocol (BSE).

Functionality: $\langle \delta \rangle = \text{BSE}(x, y)$, where $\delta = 1$ if $x = y$; $\delta = 0$, otherwise
Input: P_0 inputs a binary string $x = x_{\{\ell\}}x_{\{\ell-1\}} \cdots x_{\{1\}}$, P_1 inputs a binary string $y = y_{\{\ell\}}y_{\{\ell-1\}} \cdots y_{\{1\}}$
Output: P_0 outputs $\langle \delta \rangle_0$, P_1 outputs $\langle \delta \rangle_1$
1: **At the online stage:**
2: P_0 sets $\langle x_i \rangle_0 = x_{\{i\}}$ and $\langle y_i \rangle_0 = 0$ for $i \in [1, \ell]$
3: P_1 sets $\langle x_i \rangle_1 = 0$ and $\langle y_i \rangle_1 = y_{\{i\}}$ for $i \in [1, \ell]$
4: P_0 and P_1 computes $\langle f_i \rangle \leftarrow (1 - \langle x_i \rangle - \langle y_i \rangle)^2$ for $i \in [1, \ell]$
5: P_0 and P_1 computes $\langle \delta \rangle \leftarrow \text{AND}^*(\langle f_1 \rangle, \langle f_2 \rangle, \dots, \langle f_\ell \rangle)$

TABLE I
A TOY EXAMPLE OF PLT PROTOCOL ($x = 6 = 0110_2, y = 10 = 1010_2$)

$S_x^0 = \{0111, 1\}, S_y^1 = \{101, 1\}$					
i	a_i	u_i	b_i	v_i	c_i
1	0111	1	1010*	0	0
2	101*	0	101	1	0
3	11*	0	10*	0	0
4	1	1	1	1	1
$(\delta = \sum_{i=1}^4 c_i = 1) \iff x < y$					

* The string with an asterisk means a random fake binary string, such that the flag u_i or v_i will be set as 0.

(more specifically, equality test for binary strings). Our basic idea is to make P_0 and P_1 generate x 's 0-encoding and y 's 1-encoding respectively, and then invoke the BSE protocol to execute equality tests on these encodings. According to the results of the comparison, we can check whether there is the same element between the two encodings, so as to see if x is less than y . Table I provides a toy example illustrating the PlainLessThan protocol, which is formally described in Algorithm 2.

First of all, P_0 computes the 0-encoding of x locally. Recall that the 0-encoding of x is defined as a set S_x^0 such that $S_x^0 = \{x_{\{\ell\}}x_{\{\ell-1\}} \cdots x_{\{i+1\}}1|x_{\{i\}} = 0, 1 \leq i \leq \ell\}$, and let

$$\text{zeroEnc}(x) := \{(i, a_i) | a_i \in S_x^0\}.$$

P_0 computes $\{(i, a_i)\} \leftarrow \text{zeroEnc}(x)$ and set $W_x \leftarrow \{i | \exists a \text{ s.t. } (i, a) \in \text{zeroEnc}(x)\}$. We find that a_i is an $(\ell - i + 1)$ -bit binary string when $i \in W_x$, while a_i is not assigned when $i \notin W_x$. To avoid privacy leaks about x by checking whether a_i is assigned, P_0 sets a_i as an $(\ell - i + 1)$ -bit random binary string for $i \in [1, \ell]$ s.t. $i \notin W_x$, and sets flags u_i to indicate whether a_i is really derived from S_x^0 . The above operations are done in Steps 1-3. Secondly, P_1 computes the 1-encoding of y locally in a similar manner as P_0 (shown in Steps 4-6). In particular, P_1 computes $\{(i, b_i)\} \leftarrow \text{OneEnc}^1(y)$ and sets $W_y \leftarrow \{i | \exists b \text{ s.t. } (i, b) \in \text{OneEnc}(y)\}$. P_1 also sets b_i as an $(\ell - i + 1)$ -bit random binary string for $i \in [1, \ell]$ s.t. $i \notin W_y$, and sets flags v_i to indicate whether b_i is really derived from S_y^1 .

¹ Similar to $\text{ZeroEnc}(x)$, we let $\text{OneEnc}(y) := \{(i, b_i) | b_i \in S_y^1\}$, where $S_y^1 = \{y_{\{\ell\}}y_{\{\ell-1\}} \cdots y_{\{i\}}y_{\{i+1\}} = 1, 1 \leq i \leq \ell\}$.

Algorithm 2: PlainLessThan Protocol (PLT).

Functionality: $\langle \delta \rangle = \text{PLT}(x, y)$, where $\delta = (x < y)$
Input: P_0 inputs $x \in \mathbb{Z}_p$, P_1 inputs $y \in \mathbb{Z}_p$ ($\ell = \lceil \log_2 p \rceil$)
Output: P_0 outputs $\langle \delta \rangle_0$, P_1 outputs $\langle \delta \rangle_1$
1: P_0 computes $\{(i, a_i)\} \leftarrow \text{zeroEnc}(x)$, and sets $W_x \leftarrow \{i | \exists a \text{ s.t. } (i, a) \in \text{zeroEnc}(x)\}$
2: P_0 picks an $(\ell - i + 1)$ -bit random binary string α_i : $\alpha_i \leftarrow \alpha_{i\{\ell-i+1\}} \alpha_{i\{\ell-i\}} \cdots \alpha_{i\{1\}} \in \{0, 1\}^{\ell-i+1}$, and sets $a_i = \alpha_i$ for $i \in [1, \ell]$ s.t. $i \notin W_x$
3: P_0 sets $\langle u_i \rangle_0 = 1$ for $i \in W_x$, $\langle u_i \rangle_0 = 0$ for $i \in [1, \ell]$ s.t. $i \notin W_x$, and $\langle v_i \rangle_0 = 0$ for $i \in [1, \ell]$
4: P_1 computes $\{(i, b_i)\} \leftarrow \text{oneEnc}(y)$, and sets $W_y \leftarrow \{i | \exists b \text{ s.t. } (i, b) \in \text{oneEnc}(y)\}$
5: P_1 picks an $(\ell - i + 1)$ -bit random binary string β_i : $\beta_i \leftarrow \beta_{i\{\ell-i+1\}} \beta_{i\{\ell-i\}} \cdots \beta_{i\{1\}} \in \{0, 1\}^{\ell-i+1}$, and sets $b_i = \beta_i$ for $i \in [1, \ell]$ s.t. $i \notin W_y$
6: P_1 sets $\langle v_i \rangle_0 = 1$ for $i \in W_y$, $\langle v_i \rangle_0 = 0$ for $i \in [1, \ell]$ s.t. $i \notin W_y$, and $\langle u_i \rangle_0 = 0$ for $i \in [1, \ell]$
7: P_0 and P_1 compute $\langle c_i \rangle \leftarrow \text{BSE}(a_i, b_i) \cdot \langle u_i \rangle \cdot \langle v_i \rangle$ for $i \in [1, \ell]$ in a parallel manner
8: P_0 and P_1 compute $\langle \delta \rangle \leftarrow \sum_{i=1}^{\ell} \langle c_i \rangle$

After that, P_0 and P_1 compute $\langle c_i \rangle \leftarrow \text{BSE}(a_i, b_i) \cdot \langle u_i \rangle \cdot \langle v_i \rangle$, which indicate whether there is a same element at the i -th position. At last, P_0 and P_1 compute $\langle \delta \rangle \leftarrow \sum_{i=1}^{\ell} \langle c_i \rangle$, where $\delta = 1$ iff $|S_x^0 \cap S_y^1| = 1$. We can further deduce that $\delta = 1$ iff $x < y$ according to (1).

C. LSB Computation Protocol

A least significant bit computation protocol $\text{ComputeLSB}(\langle x \rangle)$ outputs an arithmetic share of the least significant bit (LSB) of a value $x \in \mathbb{Z}_p$, where p is the modulus of the ring and the same p is used throughout this section. Let $x_{\{1\}}$ denote the LSB of x . Since x is represented by two additive shares $\langle x \rangle_0$ and $\langle x \rangle_1$, the LSB of x depends on whether the (non-modular) sum of the two shares crosses the modulus p . Formally,

$$x_{\{1\}} = \begin{cases} \langle x \rangle_{0\{1\}} \oplus \langle x \rangle_{1\{1\}}, & \text{if } \langle x \rangle_0 + \langle x \rangle_1 < p, \\ \langle x \rangle_{0\{1\}} \oplus \langle x \rangle_{1\{1\}} \oplus 1, & \text{if } \langle x \rangle_0 + \langle x \rangle_1 \geq p. \end{cases} \quad (2)$$

Based on the formula, we propose the LSB protocol as shown in Algorithm 3. Since all arithmetic operations are performed over the finite field $\mathbb{Z}_p$, reconstructing a secret-shared value $x = \langle x \rangle_0 + \langle x \rangle_1 \bmod p$ may cause a wrap-around when the non-modular sum crosses the modulus boundary. In such cases, the least significant bit (LSB) of the true value and that of the mod-reduced value differ by exactly one bit. Therefore, detecting this wrap-around condition and compensating for it is necessary to ensure correctness.

Specifically, P_0 first extracts the LSB of its share $a_0 = \langle x \rangle_{0\{1\}} = \langle x \rangle_0 \bmod 2$, and P_1 extracts $a_1 = \langle x \rangle_{1\{1\}} = \langle x \rangle_1 \bmod 2$. The parties then secret-share a_0 and a_1 to obtain the arithmetic shares $\langle a_0 \rangle$ and $\langle a_1 \rangle$, and compute

$$\langle u \rangle \leftarrow \langle a_0 \rangle + \langle a_1 \rangle - 2\langle a_0 \rangle \cdot \langle a_1 \rangle = \langle a_0 \oplus a_1 \rangle.$$

Algorithm 3: LSB Computation Protocol (ComputeLSB).

Functionality: $\langle x_{\{1\}} \rangle = \text{ComputeLSB}(\langle x \rangle)$, where $x_{\{1\}}$ is the least significant bit of x
Input: P_0 inputs $\langle x \rangle_0$, P_1 inputs $\langle x \rangle_1$
Output: P_0 outputs $\langle x_{\{1\}} \rangle_0$, P_1 outputs $\langle x_{\{1\}} \rangle_1$
1: P_0 locally extracts $a_0 = \langle x \rangle_0 \bmod 2$, P_1 locally extracts $a_1 = \langle x \rangle_1 \bmod 2$
2: P_0 and P_1 share a_0 and a_1 : P_0 sets $\langle a_0 \rangle_0 = a_0$ and $\langle a_1 \rangle_0 = 0$, P_1 sets $\langle a_0 \rangle_1 = 0$ and $\langle a_1 \rangle_1 = a_1$
3: P_0 and P_1 compute $\langle u \rangle \leftarrow \langle a_0 \rangle + \langle a_1 \rangle - 2\langle a_0 \rangle \cdot \langle a_1 \rangle$
4: P_0 and P_1 compute $\langle v \rangle \leftarrow 1 - \text{PLT}(\langle x \rangle_0, p - \langle x \rangle_1)$, where p is the modulus of the ring
5: P_0 and P_1 compute $\langle x_{\{1\}} \rangle \leftarrow \langle u \rangle + \langle v \rangle - 2\langle u \rangle \cdot \langle v \rangle$

Next, P_0 and P_1 invoke the PlainLessThan protocol to detect whether a wrap-around occurs, i.e., whether $\langle x \rangle_0 + \langle x \rangle_1 \geq p$:

$$\langle v \rangle \leftarrow 1 - \text{PLT}(\langle x \rangle_0, p - \langle x \rangle_1),$$

where $v = 0$ indicates no wrap-around and $v = 1$ otherwise. If a wrap-around occurs, the LSB must be flipped to recover the correct result. Accordingly, the final LSB is computed as

$$\langle x_{\{1\}} \rangle \leftarrow \langle u \rangle + \langle v \rangle - 2\langle u \rangle \cdot \langle v \rangle = \langle \langle x \rangle_{0\{1\}} \oplus \langle x \rangle_{1\{1\}} \oplus v \rangle,$$

which guarantees that the extracted LSB is consistent with the true (non-modular) arithmetic value.

D. Secure Comparison Protocol

A Secure Comparison protocol $\text{SC}(\langle x \rangle, \langle y \rangle)$ outputs $\langle \delta \rangle$ with inputting arithmetic shared values $\langle x \rangle, \langle y \rangle$ over $\mathbb{Z}_p$, where $\delta = (x \geq y)$. Since we use a two's complement to represent negative values in our content, the comparison operation can be converted to the most significant bit (MSB) computation. Concretely, $x \geq y$ if $\text{MSB}(x - y) = 0$ and $x < y$ if $\text{MSB}(x - y) = 1$. There are some researches [24], [33] on secure protocols for MSB functionality with shared inputs. An effective solution [33] uses the ripple carry adder (RCA) to extract the MSB of an ℓ -bit number, which requires communication rounds $O(\ell)$ between the two parties. Even if recent research [24] adopts the parallel prefix adder (PPA) to optimize this method, however, it still needs $O(\log \ell)$ rounds of communication.

To reduce the communication complexity, we transform the problem into the LSB computation, which is clearly much easier than computing the MSB. Our construction is based on the following formula [34]: $\text{MSB}(a) = \text{LSB}(2a)$ if we are working over an odd field. Intuitively, if $a > n/2$, then $2a$ will wrap around modulo n , resulting in an odd value. Hence, the least significant bit of $2a$ is 1. Conversely, if $a \leq n/2$, $2a \bmod n$ remains even, and LSB is 0. This equivalence allows us to efficiently determine the MSB in secure computation by extracting a single least significant bit.

Based on the above ideas, we construct a secure and round-efficient comparison protocol that is built on the ComputeLSB protocol. The SC protocol proceeds as follows:

- 1) P_0 and P_1 compute $\langle t \rangle \leftarrow \langle x \rangle - \langle y \rangle$.

2) P_0 and P_1 invoke the `ComputeLSB` protocol to compute $\langle \delta \rangle \leftarrow 1 - \text{ComputeLSB}(2 \langle t \rangle)$.

The `SC` protocol outputs $\langle \delta \rangle_0$ and $\langle \delta \rangle_1$, and its correctness can be shown by verifying that the reconstructed results $\delta = (x \geq y)$, which is indeed correct since $\delta = 1 - \text{LSB}(2t) = 1 - \text{MSB}(t) = 1 - \text{MSB}(x - y) = (x \geq y)$.

Round-by-round schedule of `SecureCompare`. The 4-round communication complexity of `SecureCompare` directly follows from the structure of the underlying `ComputeLSB` protocol. We summarize the round-by-round execution below.

Round 1. The two servers jointly compute the value $\langle u \rangle = \langle a_0 \rangle + \langle a_1 \rangle - 2\langle a_0 \rangle \cdot \langle a_1 \rangle$, which involves a single secure multiplication. This step consumes one communication round.

Rounds 2–4. In parallel with the computation of $\langle u \rangle$, the servers invoke `PlainLessThan` to detect whether a modular wrap-around occurs. `PlainLessThan` itself consists of three communication rounds, during which all bitwise equality tests and multiplications are fully parallelized.

Final output. After both $\langle u \rangle$ and the wrap-around indicator $\langle v \rangle$ are available, the final LSB $\langle x_{\{1\}} \rangle$ is computed using one additional secure multiplication. This step is merged into the last round of `PlainLessThan`, resulting in a total of four communication rounds.

As `SecureCompare` only performs local arithmetic before and after invoking `ComputeLSB`, it inherits the same 4-round complexity. The key round savings come from parallelizing the secure multiplication and the `PlainLessThan` invocation, and from avoiding bit-decomposition and carry-propagation steps required by MSB-based comparison protocols.

VI. SECURE DBSCAN CLUSTERING

In this section, we present secure DBSCAN clustering based on the communication-efficient secure comparison protocol. Our scheme consists of three phases: secure data outsourcing, secure neighborhood query, and secure outsourced clustering.

A. Secure Data Outsourcing

Our protocols proceed in the odd field $\mathbb{Z}_p$ so that all data are required to be integers. For the floating-point numbers, we could convert the numbers into integers by multiplying the numbers by a large constant, e.g., 10^ϕ (ϕ is the desired number of decimal places). This solution is extremely pervasive in secure machine learning applications (e.g., [24], [34], [35]), which is proven to be correct in [36]. To perform the bit-wise operations, we use a two's complement to represent the negative values. The most significant bit (MSB) indicates the sign, such that $\text{MSB}(x) = 1$ when x is a negative value and $\text{MSB}(x) = 0$ otherwise.

Range and Overflow Considerations: To ensure correctness, we require that the accumulated distance values do not wrap around modulo p , so that modular arithmetic preserves the true ordering of distances. Let each coordinate value lie in a bounded range $|x[j]| \leq B$ after fixed-point scaling, and let m denote the data dimension. Then the squared Euclidean distance satisfies $\sum_{j=1}^m (x[j] - y[j])^2 \leq 4mB^2$. We choose the prime modulus p such that $4mB^2 < p$, which guarantees that all intermediate distance values are strictly smaller than p and thus correctly

```
class InputElement{
    clusterId = null,    % the id of the cluster that the element belongs to
    isCorePoint = 0,    % indicates whether the element is a core point
    Vector neighbors = null %* the results of the comparison of the
                           element's distances to all other elements */
}
```

Fig. 2. Structure of the `InputElement` Class.

represented in $\mathbb{Z}_p$ without overflow. This condition is standard in secure fixed-point arithmetic and can be satisfied by selecting an appropriate scaling factor and modulus size.

Based on the above data representation, the data owners additively share their data points among each of the two servers. Specifically, for a point $p_i = (p_i[1], \dots, p_i[m])$ (m denotes the dimension of the point), a set of random numbers $r_j \in \mathbb{Z}_p$ ($1 \leq j \leq m$) are chosen, and then the data owner sets $\langle p_i \rangle_0 = [r_1, \dots, r_m]$ and computes $\langle p_i \rangle_1 = [p_i[1] - r_1, \dots, p_i[m] - r_m]$. Then, one share $\langle p_i \rangle_0$ is uploaded to Server S_0 and another share $\langle p_i \rangle_1$ is uploaded to Server S_1 . Assume that there are N secret-shared points dispatched to cloud servers from multiple data owners, denoted as $\langle P \rangle = \{\langle p_1 \rangle, \dots, \langle p_N \rangle\}$, where $\langle p_i \rangle = (\langle p_i[1] \rangle, \dots, \langle p_i[m] \rangle)$. Moreover, the two cloud servers are also supplied with secret-shared DBSCAN's parameters $\langle \epsilon \rangle$ and $\langle MinPt \rangle$.

B. Secure Neighborhood Query

With the shares of the data owners' points and DBSCAN's parameters at the server, a secure neighborhood query is then performed, which produces the shares of the indicators that designate the ϵ -neighborhood (cf. Definition 1) for each point. Euclidean distance (ED) is recognized as one of the most important and frequently used metrics for clustering algorithms. In the original plaintext DBSCAN algorithm, the neighborhood query of every point involves a lot of pairwise ED calculations and the query is executed many times in different loops.

To reduce the overhead of the neighborhood queries, we define a `InputElement` class as shown in Fig. 2. The attributes of the class include `clusterId`, `isCorePoint`, and `neighbors`, where the id of the cluster that the element belongs to is stored in `clusterId`, `isCorePoint` indicates whether the element is a core point, and the results of the comparison of the element's distances to all other elements are stored in `neighbors`. The servers create an object of `InputElement` for every input point and hold the shares of the object's attributes' values. To determine if two elements p_1, p_2 are neighbors, their Euclidean distance $ED(p_1, p_2) = \sqrt{\sum_{j=1}^m (p_1[j] - p_2[j])^2}$ has to be smaller than or equal to ϵ . We convert the comparison into $\sum_{j=1}^m (p_1[j] - p_2[j])^2 \leq \epsilon^2$ for the avoidance of square root calculations over the secret-shared data. We denote the set of all initialized elements as `element_list`, and its secret-shared form is denoted as $\langle \text{element_list} \rangle$ in which the attributes' values are shared between S_0 and S_1 . As shown in Algorithm 4, S_0 and S_1 calculate all distances and initialize the attributes for all elements in a parallel fashion.

Algorithm 4: Element Initialization Protocol.

Input: S_0 and S_1 input the shared point set $\langle P \rangle = \{\langle p_1 \rangle, \dots, \langle p_N \rangle\}$, the shared parameters $\langle \epsilon^2 \rangle, \langle MinPt \rangle$
Output: S_0 and S_1 output $\langle element_list \rangle$, in which each element's $isCorePoint$, $neighbors$ are assigned

- 1: S_0 and S_1 :
- 2: **for** $i \in [1, N]$ **do in parallel**
- 3: $\langle d \rangle \leftarrow \sum_{j=1}^m (\langle p_1[j] \rangle - \langle p_i[j] \rangle)^2$
- 4: $\langle \delta_i \rangle \leftarrow SC(\langle \epsilon^2 \rangle, \langle d \rangle)$ % $\delta_i = (ED(p_1, p_i) \leq \epsilon)$
- 5: **end for**
- 6: $\langle element_list[1].isCorePoint \rangle \leftarrow SC(\sum_{i=1}^n \langle \delta_i \rangle, \langle MinPt \rangle)$
- 7: $\langle element_list[1].neighbors \rangle = [\langle \delta_1 \rangle, \dots, \langle \delta_N \rangle]$
- 8: Repeat Steps 2-7 until all the elements are initialized

C. Secure Outsourced Clustering

With the secret-shared results of the neighborhood query, secure outsourced clustering is then performed, which produces the shares of $clusterId$ indicating the cluster that the element belongs to. To implement an information-theoretically secure outsourced clustering, the two servers should learn nothing about the input beyond the output. This security criterion implies that our scheme must not only execute the neighborhood query securely, but also ensure no intermediate information (such as the number and size of clusters) is leaked in the procedure of clustering. The main challenge here is that the clustering algorithm should be data-oblivious, meaning that its execution path should not depend on the input. Obviously, the original plaintext DBSCAN (cf. Algorithm 7) is realized with a queue whose execution path is directly related to the input order, so we cannot transform the original DBSCAN into a secure clustering protocol trivially. We note that an existing mechanism [10] fulfills the above security requirement, but suffers heavy computation and communication overhead, especially requiring $O(N^3)$ round complexity in the number of points N . To ensure both security and efficiency, we first propose a new plaintext DBSCAN algorithm tailored for high-efficiency cryptographic computation, then convert it into a secure form with lightweight secret-sharing-based primitives.

Crypto-friendly DBSCAN Algorithm: Algorithm 5 presents our proposed plaintext DBSCAN algorithm, which serves as the underlying algorithm for our SEO-DBSCAN. Distinctly for security considerations, this algorithm is realized with a double-loop structure. This design is to avoid using a queue, which has the potential to leak data access patterns. Before diving into the algorithmic details, it is necessary to define some notations and functions. Here, $element_list[i]$ refers to the i -th element of the set, while $neighbors[i]$ denotes the i -th value of the vector. We introduce two primary functions: *CombinedIsCorePoint* and *OneCount*. The *CombinedIsCorePoint* function takes a set of *InputElement* as its input, combining their *isCorePoint*-values into a vector form with N values. On the other hand, the *OneCount* function processes a vector, aiming to count how many values 1 are contained within it.

Algorithm 5: Crypto-friendly DBSCAN Algorithm.

Input: $element_list$ (after initialization)
Output: $element_list$ (in which each element's $clusterId$ is determined)

- 1: $CurrentClusterID = 0$
- 2: $sIsCorePoint \leftarrow CombinedIsCorePoint(element_list)$
- 3: **for** $i \in [1, N]$ **do**
- 4: $sNeighborAreCoreElement \leftarrow element_list[i].neighbors \wedge sIsCorePoint$
- 5: $element_list[i].clusterId = -1$
- 6: **for** $k \in [1, N]$ **do**
- 7: $sDensityConnected \leftarrow sNeighborAreCoreElement \wedge element_list[k].neighbors$
- 8: **if** $OneCount(sDensityConnected) \geq 1$ **then**
- 9: $element_list[i].neighbors[k] = element_list[k].neighbors[i] = 1$
- 10: **if** $element_list[i].isCorePoint$ **then**
- 11: $element_list[k].clusterId \leftarrow CurrentClusterID$
- 12: **end for**
- 13: $CurrentClusterID \leftarrow CurrentClusterID + 1$
- 14: **end for**
- 15: **return** $element_list$

The algorithm begins with the input of all initialized elements, labeled as $element_list$. Initially, we set $CurrentClusterID = 0$, and invoke the *CombinedIsCorePoint* function to get a vector $sIsCorePoint$ with inputting $element_list$. To achieve clustering, two nested looping statements are employed to assign the $clusterId$ for each element in $element_list$. In the outer for-loop, by performing a bitwise AND operation ($\wedge$) on the two vectors, $element_list[i].neighbors$ and $sIsCorePoint$, we obtain the vector $sNeighborAreCoreElement$, which records the core points among the neighbors of the element $element_list[i]$. Then, we initialize $element_list[i].clusterId$ as -1 , marking it as a noise point. Next, through the nested for-loop, we also traverse through all elements in $element_list$ to find the points that are density-connected to the element in the outer for-loop, and record them into both elements' $neighbors$, i.e., set $element_list[i].neighbors[k] = element_list[k].neighbors[i] = 1$ (shown in lines 7 to 9). Furthermore, in the nested for-loop, we set $element_list[k].clusterId = CurrentClusterID$ only when the element in the outer for-loop is a core point (shown in lines 10 and 11). This is because if the element in the outer for-loop is a border point (i.e., not a core point), it may appear in more than one cluster, such that the elements belonging to different clusters will be given the same $clusterId$. At last, all the density-connected points are given the same $clusterId$ after the doubly nested loops.

Secure Outsourced DBSCAN Protocol: Building upon the plaintext DBSCAN algorithm, we design a secure and

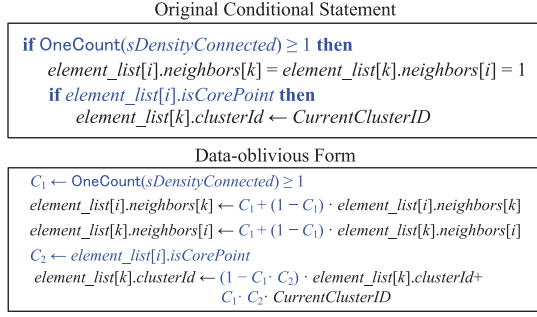


Fig. 3. Illustration of data-oblivious design for the conditional statements.

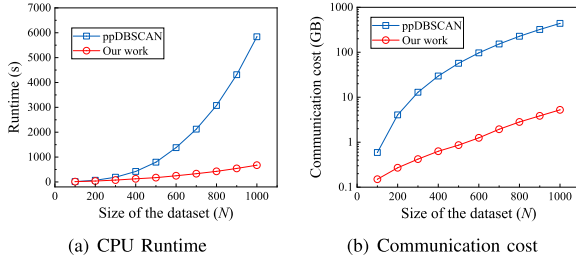


Fig. 4. Performance comparison of our work with ppDBSCAN [10].

communication-efficient outsourced DBSCAN protocol. This refined protocol allows for $O(N^2)$ round complexity between servers. Our insight is to execute all the arithmetic operations of crypto-friendly DBSCAN with arithmetic secret sharing, and all the comparison operations with our SC protocol. Moreover, the conditional statements need to be executed in a data-oblivious manner. We tackle this challenge by introducing two variables, C_1 and C_2 , which enable the transformation of conditional statements into sequential ones. Our data-oblivious design for the conditional statements is illustrated in Fig. 3.

Importantly, all control-flow-dependent variables, including $\langle \text{CurrentClusterID} \rangle$, are maintained and updated entirely in secret-shared form. The increment of $\langle \text{CurrentClusterID} \rangle$ is performed deterministically in each iteration of the fixed outer loop, which always runs for N rounds, regardless of whether a new cluster is actually formed. Cluster assignments are conditionally applied only through secret-shared predicates (i.e., $\langle C_1 \rangle$ and $\langle C_2 \rangle$), ensuring that the observable computation trace remains identical across all iterations. As a result, the two cloud servers cannot infer any implicit information, such as the number of clusters, from the evolution of $\langle \text{CurrentClusterID} \rangle$ or from the protocol execution. For a comprehensive understanding, we delve into the specifics of the secure outsourced DBSCAN protocol in Algorithm 6.

VII. THEORETICAL ANALYSIS

A. Correctness Analysis

Since our secure DBSCAN protocol is implemented by using the crypto-friendly DBSCAN algorithm (cf. Algorithm 5) as the underlying scheme, we should prove the correctness of

Algorithm 6: Secure Outsourced DBSCAN Protocol.

Input: $\langle \text{element_list} \rangle$
Output: $\langle \text{element_list} \rangle$ (in which each element's clusterId is determined)

- 1: $\mathcal{S}_0$ and $\mathcal{S}_1$;
- 2: $\langle \text{CurrentClusterID} \rangle \leftarrow \langle 0 \rangle$
- 3: $\langle s_{\text{IsCorePoint}} \rangle \leftarrow \text{CombinedIsCorePoint}(\langle \text{element_list} \rangle)$
- 4: **for** $i \in [1, N]$ **do**
- 5: $\langle s_{\text{NeighborAreCoreElement}} \rangle \leftarrow \langle \text{element_list}[i].\text{neighbors} \rangle \odot \langle s_{\text{IsCorePoint}} \rangle$
- 6: $\langle \text{element_list}[i].\text{clusterId} \rangle \leftarrow \langle -1 \rangle$
- 7: **for** $k \in [1, N]$ **do**
- 8: $\langle s_{\text{DensityConnected}} \rangle \leftarrow \langle \text{element_list}[k].\text{neighbors} \rangle \odot \langle s_{\text{NeighborAreCoreElement}} \rangle$
- 9: $\langle t \rangle \leftarrow \sum_{j=1}^n \langle s_{\text{DensityConnected}}[j] \rangle$
- 10: $\langle C_1 \rangle \leftarrow \text{SC}(\langle t \rangle, \langle 1 \rangle)$
- 11: $\langle \text{element_list}[i].\text{neighbors}[k] \rangle \leftarrow \langle C_1 \rangle + (1 - \langle C_1 \rangle) \cdot \langle \text{element_list}[i].\text{neighbors}[k] \rangle$
- 12: $\langle \text{element_list}[k].\text{neighbors}[i] \rangle \leftarrow \langle C_1 \rangle + (1 - \langle C_1 \rangle) \cdot \langle \text{element_list}[k].\text{neighbors}[i] \rangle$
- 13: $\langle C_2 \rangle \leftarrow \langle \text{element_list}[i].\text{isCorePoint} \rangle$
- 14: $\langle \text{element_list}[k].\text{clusterId} \rangle \leftarrow (1 - \langle C_1 \rangle \cdot \langle C_2 \rangle) \cdot \langle \text{element_list}[k].\text{clusterId} \rangle + \langle C_1 \rangle \cdot \langle C_2 \rangle \cdot \langle \text{CurrentClusterID} \rangle$
- 15: **end for**
- 16: $\langle \text{CurrentClusterID} \rangle \leftarrow \langle \text{CurrentClusterID} \rangle + 1$
- 17: **end for**
- 18: **return** $\langle \text{element_list} \rangle$

this algorithm. Let C be an arbitrary cluster returned by this algorithm. We next show that C satisfies Definition 5.

Maximality: Let p be an arbitrary core point in C , and q be any point density-reachable from p . We should prove that $q \in C$. By Definition 3, there is a chain of points $p_1, \dots, p_t$ ($t \geq 2$) such that $p_1 = p, p_t = q, p_1, \dots, p_{t-1}$ are core points, and $p_{i+1} \in N_\epsilon(p_i)$ for each $i \in [1, t-1]$. Let p_{outer} be a point that is visited in the outer for-loop and p_{nested} be a point that is visited in the nested for-loop. According to the design of the crypto-friendly DBSCAN algorithm, the two points p_{outer} and p_{nested} that are density-connected are assigned the same clusterId only when p_{outer} is a core point. It thus follows that there must be a core point $p_{\text{outer}} \in \{p_1, \dots, p_{t-1}\}$ while p and q are visited in the nested for-loop. Therefore, p and q must be assigned the same clusterId (i.e., in the same cluster).

Connectivity: Let p be an arbitrary core point in C . For any point $q \in C$, we will show that q is density-connected from p . Let p_{outer} be a point that is visited in the outer for-loop and p_{nested} be a point that is visited in the nested for-loop. Since p and q are assigned the same clusterId , there must be a core point p_{outer} while p and q are visited in the nested for-loop.

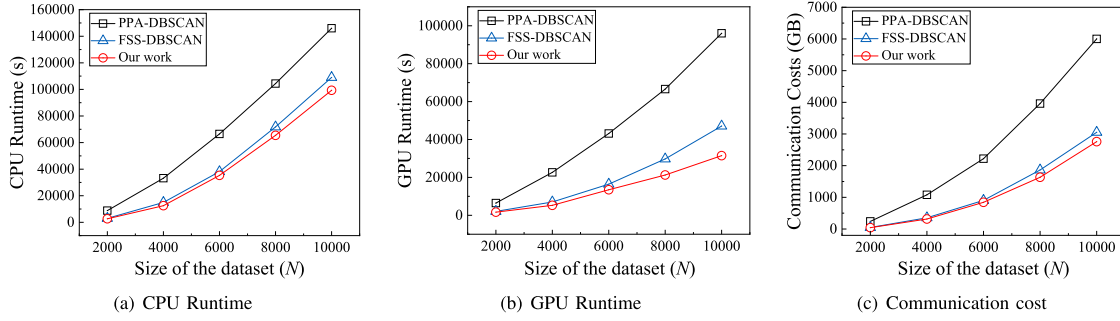


Fig. 5. Performance comparison of our work with PPA-DBSCAN [12] and FSS-DBSCAN [14].

TABLE II
NUMBER OF ROUNDS AND TOTAL COMMUNICATION OF OUR SC PROTOCOL
AND ITS SUBROUTINES (ℓ DENOTING THE BIT LENGTH OF THE DATA)

Protocol	#Round	Total comm. (elements)
BinaryStringEqual	3	6ℓ
PlainLessThan	3	$6\ell^2 + 4\ell$
ComputeLSB	4	$6\ell^2 + 4\ell + 8$
SecureCompare	4	$6\ell^2 + 4\ell + 8$

TABLE III
COMPLEXITY ANALYSIS OF OUR SEO-DBSCAN AND PRIOR ARTS (N
DENOTES THE NUMBER OF INPUT POINTS)

Phase	Scheme	Computation	Round
Neighborhood Query	ppDBSCAN [10]	$O(N^2)$	$O(N^2)$
	FSS-DBSCAN [14]	$O(N^2)$	$O(1)$
	Ours	$O(N^2)$	$O(1)$
Outsourced Clustering	ppDBSCAN [10]	$O(N^3)$	$O(N^3)$
	FSS-DBSCAN [14]	$O(N^2)$	$O(N)$
	Ours	$O(N^2)$	$O(N)$
Total	ppDBSCAN [10]	$O(N^3)$	$O(N^3)$
	FSS-DBSCAN [14]	$O(N^2)$	$O(N)$
	PPA-DBSCAN [12]	$O(N^2)$	$O(N)$
	Ours	$O(N^2)$	$O(N)$

Combining this fact with how the attribute *clusterId* is assigned the same value, we know that p and q are both density-reachable from the core point p_{outer} . By Definition 4, we can claim that p is density-connected to q .

B. Complexity Analysis

The communication and round complexities of the SC protocol and its subroutines are detailed in Table II, with ℓ denoting the bit length of the input values. For a comprehensive understanding of these complexities, refer to the detailed analysis presented in Appendix B, available online. Table III shows the computation and round complexities of our SEO-DBSCAN and prior works, including ppDBSCAN [10], PPA-DBSCAN [12], and FSS-DBSCAN [14]. We can find that the proposed SEO-DBSCAN method significantly outperforms ppDBSCAN in both the secure neighborhood query and secure outsourced clustering phases, achieving the same low complexity as FSS-DBSCAN. In terms of overall complexity, SEO-DBSCAN is comparable with current state-of-the-art methods, such as FSS-DBSCAN and PPA-DBSCAN. It

should be noted, however, that PPA-DBSCAN is fundamentally an approximate secure DBSCAN clustering algorithm, whose clustering quality heavily depends on hyperparameter selection. Accurately determining suitable hyperparameters in encrypted settings is particularly challenging, thus potentially limiting its practical applicability. Although SEO-DBSCAN shares the same theoretical complexity with FSS-DBSCAN, we will demonstrate in the subsequent experimental evaluation that SEO-DBSCAN outperforms FSS-DBSCAN in practical performance.

C. Security Analysis Overview

Our security analysis follows the standard simulation-based definition of two-party computation under the semi-honest adversary model [37]. Each protocol component is shown to be secure by constructing a simulator that can reproduce the view of a corrupted party using only its local input and prescribed output. If such a simulator exists and its output is computationally indistinguishable from the adversary's real view, then the protocol reveals no additional information beyond the intended output.

We analyze the security of our protocols in a modular fashion using Canetti's composition theorem [38], which states that a protocol invoking several secure sub-protocols remains secure when each sub-protocol is replaced with an ideal functionality. In our setting, basic arithmetic operations on secret shares and the AND* protocol are already known to be secure [25], [39]. Building on these primitives, we show that BSE, PLT, ComputeLSB, and SC can all be simulated in a hybrid model where a trusted party realizes each sub-function. Since all intermediates remain in secret-shared form and messages between the two clouds are either uniform or protected via Beaver triples, the simulators can simply sample uniformly random shares to reproduce the view of a corrupted party.

Finally, by applying the composition theorem, we extend the security of the sub-protocols to the element initialization protocol and the secure outsourced DBSCAN protocol. In both cases, the only information revealed is the final output in secret-shared form, while all intermediate values (e.g., distances, core-point indicators, and cluster propagation states) remain hidden. Formal proofs of these claims are provided in Appendix C, available online.

TABLE IV
RUNTIME AND COMMUNICATION COST OF SC PROTOCOL AND ITS
SUBROUTINES

Protocol	Runtime			Comm. Cost (KB)
	Comp. time (μ s)	Comm. time (ms)	Total (ms)	
BinaryStringEqual	0.26	0.64	0.65	0.75
PlainLessThan	18.34	0.64	0.66	24.50
LSBCompute	19.43	0.73	0.75	24.54
SecureCompare	19.74	0.74	0.76	24.54

VIII. EXPERIMENT

A. Implementation and Experimental Setup

The system is implemented in Python 3.12 with the NssMPClib², which is a lightweight secure multi-party computing library based on Pytorch. In our experiments, p for the field $\mathbb{Z}_p$ is a 32-bit odd prime, following prior works [24]. Our experiments are conducted on two separate servers (as server S_0 and S_1) running Ubuntu 20.04 with a 3.7 GHz Intel Core i9-10900 k CPU, 128 GB RAM, and NVIDIA GeForce RTX 3090. Consistent with prior art (i.e., FSS-DBSCAN [14]), we evaluate our secure protocols in the LAN setting with 0.22 ms round-trip latency and 10 Gbps network bandwidth. We conduct experiments over the following public datasets: Lsun dataset contains 400 2-dimensional data points in 3 clusters, zelnik5 dataset contains 511 2-dimensional data points in 5 clusters, Longsquare dataset contains 900 2-dimensional data points in 6 clusters, and S1 dataset contains 5000 2-dimensional data points in 15 clusters.

B. Experiments for Secure Comparison Protocol and Its Subroutines

1) *Runtime and Communication Cost*: Table IV shows the runtime and communication cost of our SecureCompare protocol and its subroutines. The numbers of inputs for BinaryStringEqual, and PlainLessThan protocols are set as 32 (i.e., in Algorithm 1 & 2, $\ell = 32$). We note that all protocols' computing operations can be achieved at the microsecond level. This is because all local computations are constructed based on the secret-sharing technique, such that they do not involve any time-consuming cryptographic operation. Furthermore, the communication time of all protocols is still very small (in microseconds), and the time increases linearly with the round of communication as shown in Table II. In addition, the communication costs of all protocols are also consistent with our complexity analysis. We find that the communication time almost occupies the whole runtime, while the influence of computation time is almost ignorable. Therefore, it is crucially important to reduce the number of communication rounds for constructing efficient cryptographic protocols based on the secret-sharing technique.

2) *Comparison With Prior Art*: To demonstrate the high efficiency of our SecureCompare protocol, Table V shows the

TABLE V
PERFORMANCE COMPARISON OF OUR WORK WITH PRIOR ART FOR SECURE
COMPARISON FUNCTIONALITY

Work	#Round	Runtime (ms)			Comm. cost (KB)
		Comp. time	Comm. time	Total	
Morita et al. [25]	6	0.27	1.32	1.59	48.10
Ohata et al. [26]	3	2.10	0.66	2.76	0.08
Sonic [24]	6	0.03	1.29	1.32	0.06
ABY [11]	3	4.83	0.66	5.49	50.48
FSS-DBSCAN [14]	5	0.02	1.07	1.09	40.76
Ours	4	0.02	0.74	0.76	24.54

performance comparison of our work with prior art. Morita et al. [25] propose a constant-round secure comparison protocol using multi-fan-in gates. However, their protocol still needs 6 rounds and the communication time occupies more than 83% of the total runtime. Ohata et al. [26] uses a Beaver triple extension technique to reduce the communication rounds of the secure comparison protocol. Although this method has an advantage in communication rounds (only 3 rounds under the parameters we consider in this paper), it requires exponentially large memory and computation costs, such that the local computation takes up most of the runtime. Sonic [24] adopts a parallel prefix adder to compute the MSB of secret shared values, so it achieves a secure comparison functionality through $\log \ell + 1$ rounds in the length of values ℓ . In particular, Sonic [24] takes 6 rounds and 1.32 ms for the comparison. ABY [11] is a common SMC framework and used by ppDBSCAN [10]. Due to the intensive computations that ABY must perform, including numerous symmetric encryptions to construct circuits, it takes a significant amount of time for comparisons. Specifically, it requires a total of 5.49 ms, of which 4.83 ms is dedicated to local computation. FSS-DBSCAN [14] designs a secure comparison protocol by combining function secret sharing and arithmetic secret sharing, resulting in 5 communication rounds. Since it requires one more round than our protocol and involves significant communication overhead for FSS key transmission, its overall performance is inferior to our proposed protocol. Compared to these works, our work considers both the computation time and the communication time, achieving superior performance than prior works.

C. Experiments for Secure DBSCAN Clustering

1) *Clustering Quality*: We evaluate the clustering quality of our proposed method by comparing its performance with the corresponding plaintext DBSCAN algorithm [7], the well-known K -means [40] and Affinity Propagation (AP) [41] algorithms, as well as state-of-the-art secure DBSCAN methods such as PPA-DBSCAN [12] and FSS-DBSCAN [14]. The evaluation employs three widely-used clustering metrics: the V-measure [42], the Adjusted Rand Index (ARI) [43], and the Adjusted Mutual Information (AMI) [44], each computed against the ground truth labels. Specifically, the V-measure is an entropy-based metric that quantifies the extent to which the criteria of homogeneity and completeness are simultaneously satisfied. Both ARI and AMI measure the agreement between the clustering result and ground truth, regardless of permutations of cluster labels. All

²<https://github.com/XidianNSS/NssMPClib>

TABLE VI
CLUSTERING QUALITY COMPARISON BETWEEN OUR APPROACH, PLAINTEXT ALGORITHMS, AND EXISTING SECURE SCHEMES

Work	Lsun			zelnik5			Longsquare			S1		
	V-measure	ARI	AMI	V-measure	ARI	AMI	V-measure	ARI	AMI	V-measure	ARI	AMI
<i>K</i> -means [40],	0.544	0.440	0.542	0.673	0.504	0.671	0.866	0.798	0.865	0.982	0.988	0.988
Affinity Propagation (AP) [41]	0.743	0.528	0.741	0.648	0.310	0.639	0.734	0.416	0.727	0.825	0.913	0.858
PPA-DBSCAN [12]	1.000	1.000	1.000	1.000	1.000	1.000	0.925	0.934	0.925	0.967	0.920	0.967
FSS-DBSCAN [14]	1.000	1.000	1.000	1.000	1.000	1.000	0.940	0.949	0.939	0.976	0.978	0.976
Plaintext DBSCAN [7]	1.000	1.000	1.000	1.000	1.000	1.000	0.940	0.949	0.939	0.976	0.978	0.976
Ours	1.000	1.000	1.000	1.000	1.000	1.000	0.940	0.949	0.939	0.976	0.978	0.976

three metrics range up to a maximum of 1, indicating perfect clustering, with scores closer to 1 representing better clustering results.

We report the evaluation results on the Lsun, zelnik5, and Longsquare datasets, as their ground truth labels are publicly available. To ensure fairness in the comparisons, we set the parameter K in the K -means algorithm equal to the known number of clusters for each dataset. For the AP algorithm, parameters are selected following standard procedures outlined in [45]. Similarly, DBSCAN parameters are determined according to conventional guidelines provided in [7]. For the secure algorithms PPA-DBSCAN and FSS-DBSCAN, we configure their parameters optimally for each dataset to achieve the best possible clustering outcomes.

Table VI shows the three kinds of clustering quality scores on the Lsun, zelnik5, and Longsquare datasets. Our SEO-DBSCAN achieves exactly the same clustering quality as the plaintext DBSCAN across all datasets, demonstrating that our secure protocol faithfully preserves the accuracy of the original algorithm while providing strong privacy guarantees. Furthermore, our results are also identical to those of FSS-DBSCAN on all datasets, indicating that both protocols achieve optimal clustering quality under secure settings. In contrast, K -means and AP exhibit significantly lower scores, particularly on datasets with irregular cluster shapes or noise (e.g., zelnik5 and Longsquare), reflecting their known limitations in handling non-convex clusters and robustness to outliers.

Due to page limits, the clustering result images are provided in Appendix D, available online, where the performance gap is also clearly illustrated. While PPA-DBSCAN also achieves high clustering quality on most datasets, its performance slightly lags behind on Longsquare and S1, where the clustering quality is more sensitive to hyperparameter tuning. Since selecting optimal parameters over encrypted data is inherently difficult, this can negatively impact its practical performance.

2) *Runtime and Communication Cost*: To demonstrate the practicality and efficiency of our secure DBSCAN clustering framework, we evaluate the performance of SEO-DBSCAN on a simulated two-dimensional dataset. As previously described, the protocol consists of three main phases: secure data outsourcing, secure neighborhood query (SNQ), and secure outsourced clustering (SOC). In the secure data outsourcing phase, the data owner applies arithmetic secret sharing to the raw data. This operation is highly efficient and completes the sharing of 20,000 two-dimensional data points in approximately 10 milliseconds. Notably, the data owner is not involved in any

subsequent computation during the SNQ or SOC phases, which highlights the lightweight nature of the protocol from the client's perspective. This design significantly enhances the scalability and usability of our scheme in practical settings. Table VII presents the runtime and communication overhead of SEO-DBSCAN across varying dataset sizes, ranging from 2,000 to 20,000 data points. Experiments were conducted under both CPU and GPU execution environments, and the total cost is decomposed into the SNQ and SOC phases to provide detailed insight into the performance characteristics of each stage.

Runtime: As the dataset size increases, the total runtime also grows accordingly. Under CPU execution, processing a dataset with 20,000 data points takes approximately 254,376 seconds, with the majority of time consumed by the SOC phase, which is consistent with its quadratic computational complexity. When GPU acceleration is employed, the runtime is significantly reduced. For the same dataset size, the total runtime drops to 79,343 seconds, yielding approximately a $3.2\times$ speedup. The advantage of the GPU is especially evident in the SNQ phase, where distance computations and comparisons benefit greatly from parallelism.

Communication Cost: In terms of communication, both SNQ and SOC phases exhibit rapidly increasing overhead as the dataset grows, showing a clear quadratic growth pattern. When processing 20,000 data points, the total communication volume reaches 10.8 TB, with the SOC phase accounting for more than half of it (approximately 6.7 TB). This result aligns with our theoretical complexity analysis and highlights the importance of optimizing communication rounds and bandwidth usage in the design of secure clustering protocols.

Overall, the experimental results demonstrate that SEO-DBSCAN can securely support datasets with tens of thousands of points while maintaining strong scalability. With GPU acceleration alleviating computational bottlenecks, our protocol remains practical and efficient in high-throughput cloud environments.

D. Comparison With Prior Art

1) *Comparison With ppDBSCAN*: We first compare our SEO-DBSCAN protocol with ppDBSCAN [10], a private density-based clustering scheme that achieves full privacy. Their experiments were conducted using Intel Core i9-7960X CPUs running at 2.8 GHz with 128 GB RAM in a LAN environment with 10 Gbit/s bandwidth. Since the experimental setup of ppDBSCAN closely aligns with ours, we adopt their reported

TABLE VII
RUNTIME AND COMMUNICATION COST OF SEO-DBSCAN

Datsize	CPU Runtime (s)			GPU Runtime (s)			Communication Cost (GB)		
	SNQ	SOC	Total	SNQ	SOC	Total	SNQ	SOC	Total
2000	413	2222	2635	201	1442	1643	16.3	25.7	42
4000	1954	10580	12534	649	4586	5235	122.4	192.6	315
6000	5723	29601	35324	1625	11829	13454	330.1	512.9	843
8000	9843	55591	65434	2574	18670	21244	641.4	992.6	1634
10000	15924	83421	99345	3896	27559	31455	1004.2	1753.8	2758
15000	26434	139861	166295	6408	45624	52032	2236.7	3808.3	6045
20000	41362	213014	254376	9838	69505	79343	4195.6	6856.4	11052

SNQ denotes Secure Neighborhood Query, and SOC denotes Secure Outsourced Clustering.

TABLE VIII
PERFORMANCE COMPARISON ON REAL-WORLD DATASETS: SEO-DBSCAN Vs. ppDBSCAN [10], PPA-DBSCAN [12], AND FSS-DBSCAN [14].

Dataset	CPU time (s)				Communication Costs (GB)			
	ppDBSCAN	PPA-DBSCAN	FSS-DBSCAN	SEO-DBSCAN	ppDBSCAN	PPA-DBSCAN	FSS-DBSCAN	SEO-DBSCAN
Lsun	421	612	91	125	29.68	9.6	0.4	0.6
zelnik5	805.65	1054	212	189	60.43	36.4	0.9	0.9
Longsquare	4313.36	3136	716	543	321.15	90.4	6.5	3.9
S1	620912.7	51360	18031	16233	>47000	1500	726	642

results for performance comparison. It is worth noting that ppDBSCAN achieves full privacy through a triple-loop structure, resulting in an overall round complexity of $O(N^3)$. This makes the protocol impractical for large-scale datasets. To mitigate the high computational overhead, ppDBSCAN introduces an upper bound on the number of iterations in the second loop, which significantly improves execution speed. However, this optimization leads to unintended privacy leakage, as the number of iterations is directly related to the distribution of the input data [10]. Consequently, the privacy guarantees of ppDBSCAN are weaker than those of our protocol.

Fig. 4 illustrates the performance comparison between our SEO-DBSCAN scheme and ppDBSCAN in terms of runtime and communication cost as the dataset size increases. Note that since ppDBSCAN only reports runtime under a CPU environment, all the results shown in Fig. 4 are also measured under CPU settings. The detailed experimental data are provided in Table X in Appendix E, available online. As shown in Fig. 4(a), the runtime of ppDBSCAN grows rapidly with the dataset size, exhibiting a cubic trend due to its $O(N^3)$ complexity. In contrast, our method demonstrates significantly lower runtime with a much flatter growth curve, consistent with its quadratic complexity. For instance, when processing a dataset of 1000 points, SEO-DBSCAN achieves an $8.65\times$ speedup over ppDBSCAN (675s vs. 5839.50s), with similar improvements across other scales (e.g., $7.23\times$ faster at 800 points).

Similarly, Fig. 4(b) shows that the communication cost of ppDBSCAN escalates sharply, reaching over 438 GB for 1000 data points. In comparison, SEO-DBSCAN requires only 5.24 GB, reducing communication by approximately $83.7\times$. These results confirm that our protocol is not only more computationally efficient but also significantly more communication-friendly, making it highly scalable for large-scale secure clustering tasks.

2) *Comparison With PPA-DBSCAN and FSS-DBSCAN:* Fig. 5 illustrates a comprehensive performance comparison between our SEO-DBSCAN protocol and two state-of-the-art schemes, PPA-DBSCAN [12] and FSS-DBSCAN [14], across varying dataset sizes. The detailed experimental data are provided in Table IX in Appendix E, available online. As shown in Fig. 5(a), SEO-DBSCAN consistently achieves the lowest CPU runtime, outperforming both FSS-DBSCAN and the significantly slower PPA-DBSCAN. In the GPU runtime comparison (Fig. 5(b)), our method again demonstrates superior efficiency, maintaining lower execution times with a more gradual growth rate as the dataset size increases. Fig. 5(c) further highlights the communication advantages of our scheme, with SEO-DBSCAN incurring the least communication overhead among the three, particularly as the dataset scales. These results collectively confirm that SEO-DBSCAN achieves better scalability, faster execution, and reduced communication cost, making it more practical for large-scale secure clustering applications.

3) *Evaluation on Real-World Datasets:* Table VIII presents a comparative performance analysis of SEO-DBSCAN against ppDBSCAN, PPA-DBSCAN, and FSS-DBSCAN on four real-world datasets: Lsun, zelnik5, Longsquare, and S1. The results highlight the superior efficiency of SEO-DBSCAN in both CPU runtime and communication cost. On smaller datasets such as Lsun and zelnik5, SEO-DBSCAN demonstrates competitive performance, achieving 125 seconds and 189 seconds runtime respectively, which is significantly faster than ppDBSCAN and comparable to FSS-DBSCAN. On larger datasets such as Longsquare and S1, the advantage becomes even more evident. SEO-DBSCAN completes in 543 seconds and 16,233 seconds respectively, while ppDBSCAN becomes impractical, taking more than 620,000 seconds on S1. In terms of communication cost, SEO-DBSCAN consistently incurs the lowest or near-lowest overhead. For example, on the S1 dataset, it reduces communication by over 30% compared to FSS-DBSCAN and

significantly outperforms both ppDBSCAN and PPA-DBSCAN. These results demonstrate that SEO-DBSCAN offers a well-balanced solution, achieving both strong scalability and low communication overhead on real-world datasets.

IX. CONCLUSION

In this paper, we present SEO-DBSCAN, a secure and efficient outsourced DBSCAN clustering framework that achieves plaintext-equivalent clustering quality, strong privacy guarantees, and state-of-the-art performance. At the core of our design is a constant-round secure comparison protocol based entirely on lightweight arithmetic secret sharing, enabling efficient and parallelizable neighborhood queries. We further craft a cryptofriendly DBSCAN variant optimized for secure computation, allowing our scheme to scale quadratically with the input size while preserving full privacy. Extensive experiments on both simulated and real-world datasets validate the practicality of our design. Compared to ppDBSCAN, SEO-DBSCAN achieves up to $8.65\times$ speedup and $83.7\times$ communication reduction. It also consistently outperforms FSS-DBSCAN and PPA-DBSCAN in runtime and bandwidth, reducing communication by over 30% on large datasets. These results highlight SEO-DBSCAN as a scalable and secure solution for density-based clustering in real-world outsourcing scenarios.

REFERENCES

- [1] P. Berkhin, "A survey of clustering data mining techniques," in *Grouping Multidimensional: Data Recent Advances in Clustering*. Berlin, Germany: Springer, 2006, pp. 25–71.
- [2] D. Liu, E. Bertino, and X. Yi, "Privacy of outsourced k-means clustering," in *Proc. 9th ACM Symp. Inf. Comput. Commun. Secur.*, 2014, pp. 123–134.
- [3] W. Wu, J. Liu, H. Wang, J. Hao, and M. Xian, "Secure and efficient outsourced k-means clustering using fully homomorphic encryption with ciphertext packing technique," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 10, pp. 3424–3437, Oct. 2021.
- [4] A. Jäschke and F. Armknecht, "Unsupervised machine learning on encrypted data," in *Proc. Int. Conf. Sel. Areas Cryptogr.*, 2018, pp. 453–478.
- [5] H.-J. Kim and J.-W. Chang, "A privacy-preserving k-means clustering algorithm using secure comparison protocol and density-based center point selection," in *Proc. IEEE 11th Int. Conf. Cloud Comput.*, 2018, pp. 928–931.
- [6] P. Mohassel, M. Rosulek, and N. Trieu, "Practical privacy-preserving k-means clustering," in *Proc. Privacy Enhancing Technol.*, 2020, vol. 2020, no. 4, pp. 414–433.
- [7] M. Ester et al., "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. Int. Conf. Knowl. Discov. Data Mining*, vol. 96, no. 34, 1996, pp. 226–231.
- [8] M. S. Rahman, A. Basu, and S. Kiyomoto, "Towards outsourced privacy-preserving multiparty DBSCAN," in *Proc. IEEE 22nd Pacific Rim Int. Symp. Dependable Comput.*, 2017, pp. 225–226.
- [9] N. Almutairi, F. Coenen, and K. Dures, "Secure third party data clustering using φ data: Multi-user order preserving encryption and super secure chain distance matrices (best technical paper)," in *Proc. Int. Conf. Innov. Techn. Appl. Artif. Intell.*, 2018, pp. 3–17.
- [10] B. Bozdemir, S. Canard, O. Ermiş, H. Mllering, and T. Schneider, "Privacy-preserving density-based clustering," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2021, pp. 658–671.
- [11] D. Demmler, T. Schneider, and M. Zohner, "ABY-A framework for efficient mixed-protocol secure two-party computation," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2015, pp. 1–15.
- [12] J. Fu, K. Cheng, Z. Chang, and Y. Shen, "PPA-DBSCAN: Privacy-preserving ρ -approximate density-based clustering," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5324–5340, Nov./Dec. 2024.
- [13] J. Gan and Y. Tao, "DBSCAN revisited: Mis-claim, un-fixability, and approximation," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2015, pp. 519–530.
- [14] J. Fu, K. Cheng, A. Song, Y. Xia, Z. Chang, and Y. Shen, "FSS-DBSCAN: Outsourced private density-based clustering via function secret sharing," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 7759–7773, 2024.
- [15] A. Amirbekyan and V. Estivill-Castro, "Privacy preserving DBSCAN for vertically partitioned data," in *Proc. Intell. Secur. Inf., IEEE Int. Conf. Intell. Secur. Inform.*, San Diego, CA, USA, 2006, pp. 141–153.
- [16] D. Jiang, A. Xue, S. Ju, W. Chen, and H. Ma, "Privacy-preserving DBSCAN on horizontally partitioned data," in *Proc. IEEE Int. Symp. IT Med. Educ.*, 2008, pp. 1067–1072.
- [17] J. Liu, J. Z. Huang, J. Luo, and L. Xiong, "Privacy preserving distributed DBSCAN clustering," in *Proc. Joint EDBT/ICDT Workshops*, 2012, pp. 177–185.
- [18] I. V. Anikin and R. M. Gazimov, "Privacy preserving DBSCAN clustering algorithm for vertically partitioned data in distributed systems," in *Proc. Int. Siberian Conf. Control Commun.*, 2017, pp. 1–4.
- [19] A.C.-C. Yao, "How to generate and exchange secrets," in *Proc. Foundations Comput. Sci.*, 1986, pp. 162–167.
- [20] T. Veugen, F. Blom, S. J. de Hoogh, and Z. Erkin, "Secure comparison protocols in the semi-honest model," *IEEE J. Sel. Topics Signal Process.*, vol. 9, no. 7, pp. 1217–1228, Oct. 2015.
- [21] T. Nishide and K. Ohta, "Multiparty computation for interval, equality, and comparison without bit-decomposition protocol," in *Proc. Public Key Cryptogr., 10th Int. Conf. Pract. Theory Public-Key Cryptogr.*, 2007, pp. 343–360.
- [22] O. Catrina and S. De Hoogh, "Improved primitives for secure multiparty integer computation," in *Proc. Secur. Cryptogr. Netw.*, 2010, pp. 182–199.
- [23] Y. Zheng, H. Duan, and C. Wang, "Towards secure and efficient outsourcing of machine learning classification," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2019, pp. 22–40.
- [24] X. Liu, Y. Zheng, X. Yuan, and X. Yi, "Securely outsourcing neural network inference to the cloud with lightweight techniques," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 620–636, Jan./Feb. 2023.
- [25] H. Morita, N. Attrapadung, T. Teruya, S. Ohata, K. Nuida, and G. Hanaoka, "Constant-round client-aided secure comparison protocol," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2018, pp. 395–415.
- [26] S. Ohata and K. Nuida, "Communication-efficient (client-aided) secure two-party protocols and its application," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, 2020, pp. 369–385.
- [27] M. Atallah, M. Bykova, J. Li, K. Frikken, and M. Topkara, "Private collaborative forecasting and benchmarking," in *Proc. ACM Workshop Privacy Electron. Soc.*, 2004, pp. 103–114.
- [28] D. Beaver, "Efficient multiparty protocols using circuit randomization," in *Proc. Annu. Int. Cryptol. Conf.*, 1991, pp. 420–432.
- [29] H.-Y. Lin and W.-G. Tzeng, "An efficient solution to the millionaires' problem based on homomorphic encryption," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, 2005, pp. 456–466.
- [30] C. Guo et al., "GFS-CNN: A GPU-friendly secure computation platform for convolutional neural networks," *J. Netw. Netw. Appl.*, vol. 3, no. 2, pp. 66–72, 2023.
- [31] A. Song, J. Fu, X. Mu, X. Zhu, and K. Cheng, "L-SecNet: Towards secure and lightweight deep neural network inference," *J. Netw. Netw. Appl.*, vol. 3, no. 4, pp. 171–181, 2024.
- [32] Partisia, "Partisia platform: A platform for privacy preserving data solutions," 2025. [Online]. Available: <https://www.partisia.com/products/partisia-platform>
- [33] Y. Zheng, H. Duan, C. Wang, R. Wang, and S. Nepal, "Securely and efficiently outsourcing decision tree inference," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1841–1855, May/Jun. 2022.
- [34] S. Wagh, D. Gupta, and N. Chandran, "SecureNN: 3-party secure computation for neural network training," in *Proc. Privacy Enhancing Technol.*, 2019, vol. 2019, no. 3, pp. 26–49.
- [35] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *Proc. 29th {USENIX} Secur. Symp.*, 2020, pp. 2505–2522.
- [36] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 19–38.
- [37] O. Goldreich, *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge, U.K.: Cambridge Univ. Press, 2009.
- [38] R. Canetti, "Security and composition of multiparty cryptographic protocols," *J. Cryptol.*, vol. 13, no. 1, pp. 143–202, 2000.

- [39] F. Kerschbaum, T. Schneider, and A. Schröpfer, "Automatic protocol selection in secure two-party computations," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, 2014, pp. 566–584.
- [40] J. A. Hartigan and M. A. Wong, "Algorithm as 136: A k-means clustering algorithm," *J. Roy. Stat. Soc. Ser. C (Appl. Statist.)*, vol. 28, no. 1, pp. 100–108, 1979.
- [41] D. Dueck, "Affinity propagation: Clustering data by passing messages," Ph.D. dissertation, Grad. Dept. Elect. Comput. Eng., Univ. Toronto, Toronto, ON, Canada, 2009. [Online]. Available: <https://hdl.handle.net/1807/17755>
- [42] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure," in *Proc. Joint Conf. Empirical Methods Natural Lang. Process. Comput. Natural Lang. Learn.*, 2007, pp. 410–420.
- [43] L. Hubert and P. Arabie, "Comparing partitions," *J. Classification*, vol. 2, no. 1, pp. 193–218, 1985.
- [44] J. E. Nguyen, X. Vinh, and J. Bailey, "Information theoretic measures for clusterings comparison: is a correction for chance necessary?," in *Proc. Int. Conf. Mach. Learn.*, 2009, pp. 1073–1080.
- [45] B. J. Frey and D. Dueck, "Clustering by passing messages between data points," *Science*, vol. 315, no. 5814, pp. 972–976, 2007.



Ke Cheng received the PhD degree in computer science and technology from Xidian University, Xi'an, China, in 2022. He is currently an associate professor with the School of Computer Science and Technology, Xidian University. His research interests include data security and privacy protection.



Jian Yang received the PhD degree in cryptography from Xidian University, Xi'an, China, in 2024. He is currently a researcher with China Electronics Data Corporation. His research interests include artificial intelligence, knowledge graphs, and large language models.



Aijing Sun is currently a professor with the School of Communication and Information Engineering, Xi'an University of Posts and Telecommunications. She is currently an associate director with the Shaanxi Key Laboratory of Information Communication Network and Security, and expert member of Communications Branch, China Institute of Electronics. Her research interests include Internet of Things technology and intelligent education information technology. She was the recipient of the third prize of the Shaanxi Science and Technology Award, second prize of Shaanxi

Provincial Higher Education Science and Technology Award, first prize of Shaanxi Province Excellent Textbook, second prize of Shaanxi Provincial Higher Education Teaching Achievement Award, and 2020 Shaanxi Provincial Teaching Teacher Award due to her excellent contributions in education and research.



Xinghui Zhu received the PhD degree in computer science and technology from Xidian University, Xi'an, China, in 2023. He is currently an associate professor with the School of Computer Science and Technology, Xidian University. His research interests include data security and IoT security.



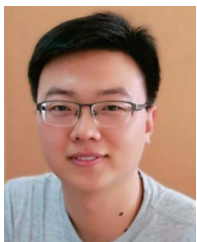
Haichang Gao (Member, IEEE) received the PhD degree in computer science and technology from Xi'an Jiaotong University, Xi'an, China, in 2006. He is currently a professor with the School of Computer Science and Technology, Xidian University. He is also in charge of a project with the National Natural Science Foundation of China. He has authored or coauthored more than 30 papers. His research interests include CAPTCHA, computer security, and machine learning.



Jiaxuan Fu received the BE and PhD degrees from Xidian University, China, in 2019 and 2025, respectively. He is currently a researcher with Tencent Inc., Shenzhen, China. His research interests include data security and machine learning security.



Yulong Shen (Senior Member, IEEE) received the BS and MS degrees in computer science and the PhD degree in cryptography from Xidian University, Xi'an, China, in 2002, 2005, and 2008, respectively. He is currently a professor with the School of Computer Science and Technology, Xidian University, where he is also an associate director of the Shaanxi Key Laboratory of Network and System Security and member of the State Key Laboratory of Integrated Services Networks. His research interests include wireless network security and cloud computing security. He has also served on the technical program committees of several international conferences, including ICEBE, INCoS, CIS, and SOWN.



Zhiwei Zhang received the PhD degree in cryptography from Xidian University, Xi'an, China, in 2019. He is currently a professor with the School of Computer Science and Technology, Xidian University. His research interests include data secure management, data storage security, and data location verification in cloud computing.